ЗАНЯТИЕ 5

Формат приложения и архитектура проекта

Окно, мобильный клиент, веб-сайт, база данных и схемы взаимодействия

16 октября 2026 • ТДИ-02-26, 10 класс

МАРШРУТ

План на 2 часа 30 минут

00:00–00:25	Форматы проекта: окно, телефон, веб-сайт
00:25–00:50	Клиент, сервер, API и базы данных
00:50–01:00	Перерыв
01:00–01:30	Какие диаграммы помогают спроектировать систему
01:30–01:50	PlantUML, draw.io и Visio
01:50–02:25	Выбор темы и построение двух схем
02:25–02:30	Проверка результата и сдача

РЕЗУЛЬТАТ

Что получится к концу занятия

Тема проекта с понятной задачей и источником данных.

Выбранный формат: окно, мобильное приложение или веб-сайт.

Диаграмма прецедентов с пользователями и их действиями.

Черновая схема модулей и обмена данными.

Предварительное решение, где хранить данные.

ОСНОВА

В любом приложении есть три задачи



Формат проекта меняет интерфейс и размещение частей, но не отменяет эти роли.

Какое окно сделать на Python

tkinter — встроенный интерфейс Python к Tcl/Tk; подходит для первой формы.

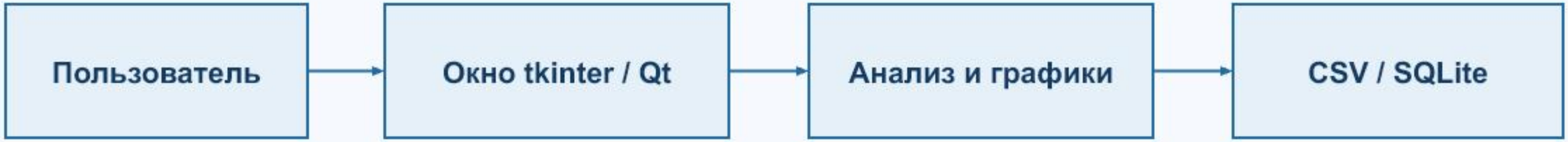
PyQt6 и PySide6 — два набора привязок Qt для более сложных окон.

PyGUI — неоднозначное имя разных проектов: проверьте конкретный пакет.

Пример: загрузить CSV, выбрать показатель и вывести график.

АРХИТЕКТУРА

Локальное оконное приложение



Всё работает на одном компьютере. Сеть и сервер пока не нужны.

Kivy: экран телефона и Python

Kivy создаёт интерфейс на Python и может работать на Android и iOS.

Офлайн вариант: экраны, логика и локальное хранилище на устройстве.

Сетевой вариант: Kivy отправляет запросы к API сервера.

Пример: ученик отмечает посещение, преподаватель видит сводку.

Flask, FastAPI и Django

Flask: небольшой веб-сайт, маршруты и шаблоны с гибкой структурой.

FastAPI: API для клиента, автоматическая схема запросов и ответов.

Django: готовые средства для моделей данных, страниц и администрирования.

Пример: открыть сайт и получить фильтруемую статистику кружка.

ПОНЯТИЕ

Что означает клиент сервер



Клиент показывает экран. Сервер выполняет общую логику и проверяет доступ.

Один запрос в клиент серверном проекте

1. Пользователь нажимает «Показать посещаемость».
2. Клиент отправляет HTTP-запрос к API, например GET /attendance.
3. Сервер проверяет запрос и читает записи из базы данных.
4. Сервер возвращает JSON с результатом.
5. Клиент рисует таблицу и график.

Пароль и прямой доступ к общей БД не размещают в клиентском приложении.

Когда нужен сервер

Один компьютер

Окно + локальная SQLite/CSV

Прототип и личный анализ данных

Меньше компонентов для запуска

Несколько пользователей

Браузер/Kivy + API + серверная БД

Роли ученика и преподавателя

Общие записи и синхронизация

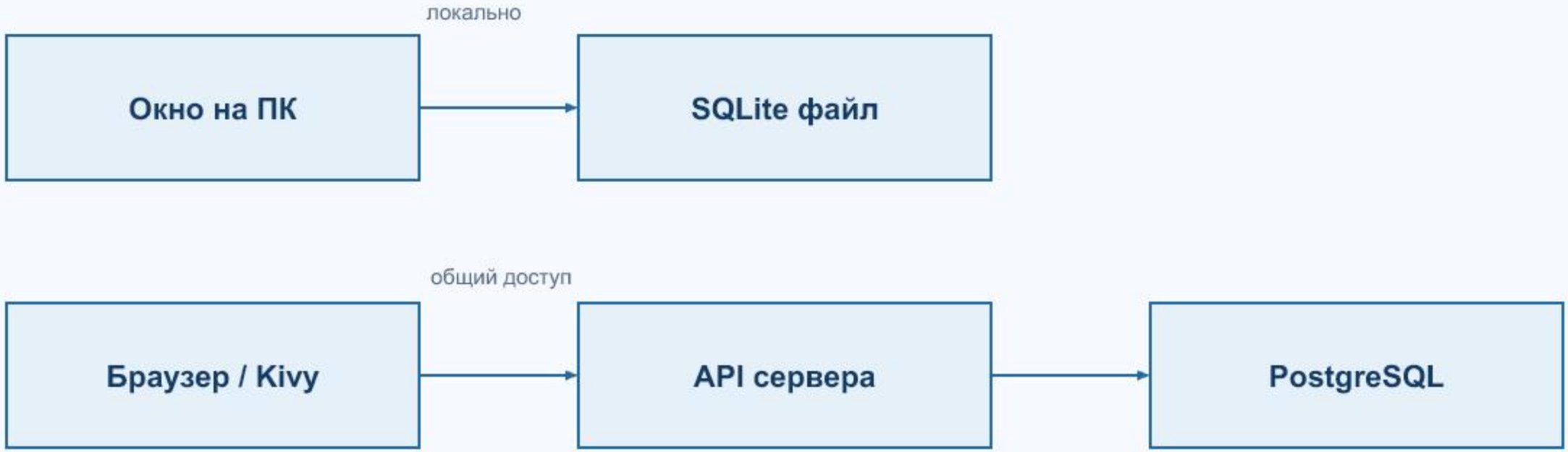
Выбирайте архитектуру по функции проекта и числу пользователей.

ХРАНЕНИЕ

SQLite, PostgreSQL, MySQL, MariaDB

СУБД	Как устроено	Пример выбора
SQLite	Файл базы рядом с приложением	Локальный прототип
PostgreSQL	Отдельный сервер БД	Общий веб-сервис
MySQL	Отдельный сервер БД	Общий веб-сервис
MariaDB	Отдельный сервер БД	Общий веб-сервис

Где может находиться база



Какие схемы обычно создают

Прецеденты: кто пользуется системой и для чего.

Компоненты / модули: какие части входят в приложение.

Последовательность: кто кому посылает сообщение и в каком порядке.

Активность / блок-схема: шаги и развилки процесса.

ER-диаграмма: сущности данных, поля и связи.

Развёртывание: на каких устройствах работают части системы.

Диаграмма прецедентов

Актор — внешняя роль: ученик, преподаватель, администратор.

Овал — цель актора: «Загрузить данные», «Посмотреть график».

Прямоугольник — граница проектируемой системы.

Линия — участие роли в прецеденте.

Здесь показывают действия пользователя, а не функции Python и таблицы БД.

ПРИМЕР

Прецеденты системы посещаемости



СТРУКТУРА

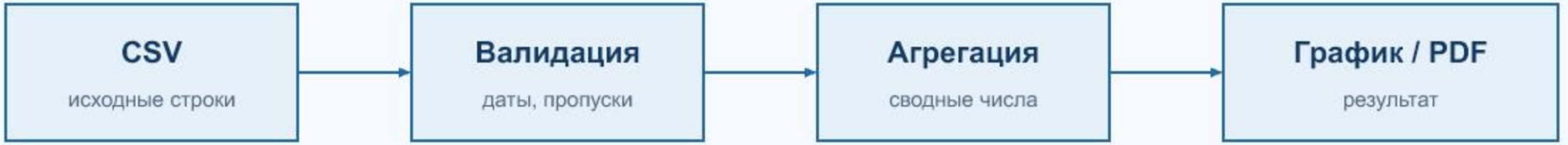
Схема модулей учебного проекта



Стрелка показывает вызов или передачу данных между частями.

ПОТОК ДАННЫХ

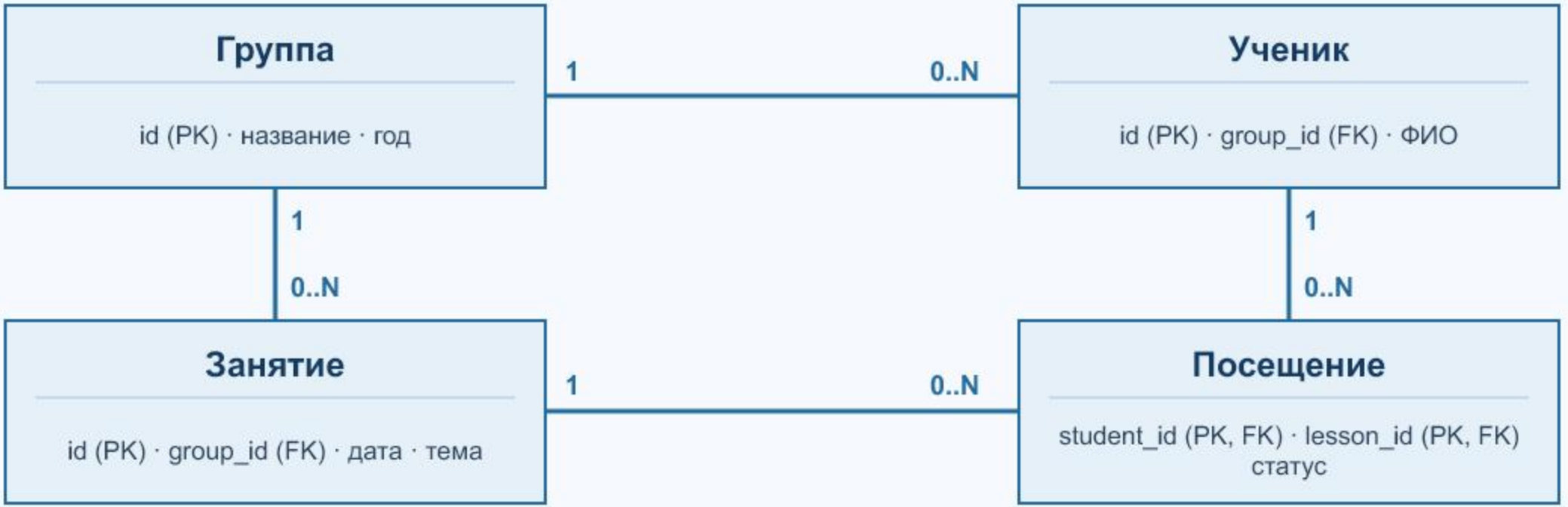
От исходного файла до отчёта



Подпишите на стрелках реальные данные: строки журнала, очищенные записи, показатели.

ЛОГИЧЕСКАЯ МОДЕЛЬ

Учёт посещаемости: сущности и связи



Пара (student_id, lesson_id) уникальна. Группа ученика и занятия должна совпадать.

Блок-схема импорта журнала



Развилка нужна там, где результат проверки меняет дальнейшее действие.

Пример диаграммы последовательности



ИНСТРУМЕНТ

PlantUML: схема из текста

Пишем текстовое описание акторов, прецедентов и связей.

PlantUML строит диаграмму из файла с исходным кодом.

Удобно хранить .puml вместе с проектом и отслеживать изменения в Git.

Не публикуйте в общедоступном редакторе личные данные из проекта.

ПРИМЕР PLANTUML

Код диаграммы прецедентов

```
@startuml
left to right direction
actor "Преподаватель" as Teacher
actor "Ученик" as Student
rectangle "Анализ посещаемости" {
  (Загрузить журнал) as Import
  (Посмотреть сводку) as View
  (Экспортировать отчёт) as Export
}
Teacher -- Import
Teacher -- View
Teacher -- Export
Student -- View
@enduml
```

draw.io и Microsoft Visio

draw.io (diagrams.net): браузерный и настольный редактор схем.

Используйте фигуры UML и сохраняйте исходный .drawio для изменений.

Visio: редактор Microsoft с шаблонами UML; доступ зависит от лицензии.

Для сдачи сохраните редактируемый исходник и экспорт PNG/PDF.

ПРАКТИКА

Тема проекта и границы первой версии

1. Назовите проблему и пользователя, которому нужен результат.
2. Укажите источник данных и два-три действия пользователя.
3. Выберите формат клиента и объясните, нужен ли сервер.
4. Выберите предварительное хранилище: файл, SQLite или серверная БД.
5. Ограничьте первую версию тремя-четырьмя основными функциями.

ПРАКТИКА

Две схемы для своей темы

Диаграмма прецедентов: 1–3 актора и 3–6 действий внутри системы.

Схема модулей: интерфейс, обработка, хранилище и нужные связи.

Если есть сервер: отдельно обозначьте клиент, API, сервер и БД.

Подпишите, какие данные идут по каждой важной стрелке.

Инструмент на выбор: PlantUML, draw.io или Visio.

Что сдать после занятия

Краткий паспорт проекта: название, задача, пользователь, данные.

Выбранный формат приложения и обоснование клиент серверной схемы.

Диаграмма прецедентов в редактируемом формате и PNG/PDF.

Черновая схема модулей с обменом данными.

Один вопрос, который нужно уточнить до начала разработки.

Проверка: акторы вне системы, действия внутри, стрелки не противоречат сценарию.